

TECHNICAL ASSESSMENT · V1.0

Engtechno

Digital Products Company

Candidate Brief · ProjectFlow

Full-Stack Software Engineer

Eng Techno | Kafr El Sheikh, Egypt

Version: 1.0 · Date: 9 September 2026 · Expected effort: 4 hours, hard cap 6 hours



Agenda

Nine parts, one repository. Read the whole brief before you start writing code — the ordering below is how the work is evaluated, not necessarily the order you should tackle it in.

- **Part One** — Understand the system
- **Part Two** — Task assignment
- **Part Three** — Activity history
- **Part Four** — Frontend implementation
- **Part Five** — Production bug
- **Part Six** — Concurrent task creation
- **Part Seven** — Testing
- **Part Eight** — Code review exercise
- **Part Nine** — Scaling question

Deliverables: source code, `README.md` , `ASSESSMENT_NOTES.md` , `BUG_REPORT.md` , `AI_LOG.md` , automated tests, schema changes.

PART ONE

Before You Start

What ProjectFlow already is, how much time to spend on it, and which technical choices are already made for you.

01 The Assessment

You will receive an existing full-stack application called **ProjectFlow** — a lightweight project-management platform where organizations manage projects, members, tasks and comments. Treat the repository as a production codebase you have recently joined. Start from github.com/engtechno/Full-Stack-Assessment-Task.

ALREADY BUILT

What exists today

- Authentication, users and organizations
- Projects and project members
- Tasks, comments and basic task management

YOUR SCOPE

What you are expected to do

- Understand the existing architecture
- Implement the requested feature
- Investigate a reported production issue
- Fix a concurrency problem, write tests
- Review a provided implementation, document decisions

Not in scope: rebuilding the application. We are measuring how you extend a system you did not write.

02 Time and Prioritization

Expected effort

4 hours

Hard cap

6 hours

Scope provided

Deliberately larger

The scope intentionally contains more opportunities for improvement than you will have time to address. Deciding what to leave alone is part of the assessment.

- What should be implemented?
- What should be improved?
- What should be left unchanged?
- What deserves testing?
- What should be documented as future work?

Bottom line: we value good prioritization over attempting to perfect everything.

03 Technology

Use the technology and architecture already provided in the repository. Follow the existing patterns unless you have a strong reason not to.

- **Additional libraries are allowed** when they solve a real problem — but avoid unnecessary dependencies.
- **Major architectural rewrites are not wanted.** Extending the system consistently scores higher than replacing parts of it.
- **Justify anything significant.** If you introduce a notable dependency or change the architecture, explain why in your notes.

PART TWO

The Work

One feature to build end to end, two production issues to investigate and fix, a test suite to extend, a pull request to review, and a scaling question to answer.

04 Understand the Existing System

Create `ASSESSMENT_NOTES.md` and document your understanding. We are interested in the overall system, not every file.

ARCHITECTURE

Questions to answer

- How is the application structured, and what are the major modules?
- Where does business logic live?
- How does the frontend talk to the backend, and how is server state handled?
- How are authentication and authorization implemented?
- How are the main entities related?

OBSERVATIONS

At least three risks or weaknesses

- What you noticed
- Why it could be a problem
- Whether you would fix it now or later
- Why

You are not required to fix all of them.

05 Task Assignment

Product has requested the ability to assign project members to tasks. A task should support an assignee.

Concept	Notes
assignee	The project member who owns the task. Nullable.
createdBy	Already exists — do not conflate the two.
project	Scopes who may be assigned at all.
status, priority	Existing fields, unchanged.

Adapt, do not transplant. Fit the assignee into the existing data model rather than reproducing a generic task shape.

06 Assignment Business Rules

RULE #1

Project membership

Only members of the project may be assigned to tasks belonging to that project.

Enforced by the backend. Frontend validation alone is not enough.

RULE #2

Assignment permissions

`OWNER`, `ADMIN` and `PROJECT_MANAGER` may assign other project members.

A regular member may assign a task to themselves, but not to anyone else.

RULE #3

Unassignment

An authorized user may remove the current assignee, and the activity history must represent that change appropriately.

Unauthorized attempts return an appropriate API error.

07 Task Activity History

Maintain an activity history for important task changes. For this assessment, track **assignee changes** only: when a task's assignee changes, an activity record is created.

```
{
  "type": "TASK_ASSIGNEE_CHANGED",
  "actor": "USER_ID",
  "task": "TASK_ID",
  "metadata": { "from": "PREVIOUS_ASSIGNEE_ID", "to": "NEW_ASSIGNEE_ID" },
  "createdAt": "..."
}
```

Your design must handle all three transitions:

- Unassigned → Assigned
- Assigned → Different user
- Assigned → Unassigned

The exact schema is yours to design, in keeping with the existing architecture and database.

08 Activity API

Create an endpoint for retrieving task activity, for example `GET /tasks/:taskId/activity`. The response contains paginated activity records. The structure may differ if the project already follows another convention.

- **Paginated.** Results must page; newest activity first.
- **Authorized.** Access control is enforced on the endpoint.
- **No obvious N+1.** Resolving actors must not cost one query per record.
- **Indexed.** Consider what the database needs to serve this query.

09 Frontend — Assignee Selector

Update the existing task details interface to support assignment. Add an assignee selector that displays project members and is searchable when appropriate.

- Loading states
- Empty states
- Error states
- Disabled states
- Permission handling
- TypeScript typing
- Responsive behavior
- Basic accessibility
- Correct update after change

Server state: optimistic updates with rollback, or another well-designed strategy. Explain significant decisions.

10 Activity Timeline UI

Add an activity section to the task details interface, reading as plain history rather than raw records.

Activity

Ammar assigned Magd	2 minutes ago
Magd changed the assignee from themselves to Ahmed	25 minutes ago
Ahmed removed the assignee	1 hour ago

Do not redesign the application. The section should integrate with the existing visual language. We are measuring your ability to extend a product consistently.

11 Production Bug Investigation

Reported by support: “Some users appear to be able to modify tasks belonging to projects they are not members of.”

Investigate and determine whether the issue exists. If you reproduce it: identify the root cause, fix it, and add regression tests. Document everything in `BUG_REPORT.md`.

- **Root Cause** — what caused it
- **Impact** — what an unauthorized user could do
- **Reproduction** — how you verified it
- **Fix** — what you changed
- **Regression Prevention** — what stops it returning
- **Or:** how you concluded it does not exist

12 Concurrent Task Creation

Tasks carry a sequential, project-specific identifier — `ENG-101`, `ENG-102`, `ENG-103`. Occasionally two tasks created at approximately the same time receive the same number.

```
const count = await Task.countDocuments({ projectId });  
  
const nextNumber = count + 1;
```

- **Investigate and fix.** The solution must stay correct when multiple application requests create tasks concurrently.
- **Document the decision.** Explain any important database or architectural choice your solution depends on.

13 Testing

Add meaningful automated tests for your implementation. At minimum, cover the important business rules.

- A project member can assign themselves
- An authorized project role can assign another project member
- A regular member cannot assign another user
- A user outside the project cannot be assigned
- Unauthorized users cannot access task activity
- Changing an assignee creates an activity record
- Unassigning a task creates the appropriate activity
- Unauthorized users cannot modify another project's tasks
- Concurrent creation cannot duplicate a task identifier

Not a coverage target. We care about what you chose to test, and why.

14 Code Review Exercise

Assume another engineer submitted this implementation. Review it as a pull request in an `ASSESSMENT_NOTES.md` section called `## Code Review`.

```
async assignTask(taskId: string, assigneeId: string, userId: string) {  
  const task = await this.taskModel.findById(taskId);  
  if (!task) { throw new NotFoundException(); }  
  
  const user = await this.userModel.findById(assigneeId);  
  if (!user) { throw new NotFoundException(); }  
  
  task.assignee = user._id;  
  await task.save();  
  return task;  
}
```

Consider correctness, security, authorization, business rules, data consistency, maintainability, error handling, performance and architecture. Explain what you would ask the engineer to change and why — do not rewrite the function for its own sake.

15 Scaling the Activity System

Assume ProjectFlow grows from roughly **5,000** users to **500,000**, and task activity becomes one of the largest datasets in the system. Explain how you would evolve your implementation — one to two pages, no more.

- Database indexes
- Cursor vs offset pagination
- Query patterns
- Data growth and retention
- Archiving
- Asynchronous processing
- Background jobs and queues
- Real-time updates
- Caching and observability

Not a technology parade. Explain what you would change, when you would change it, and why.

PART THREE

Working Practices

How you use AI tools, how you use Git, and what another engineer needs in order to run your work without asking you.

16 AI Usage Policy

AI tools are explicitly allowed — Claude Code, Codex, ChatGPT, Copilot, Cursor, Gemini, any coding assistant. Using them will not count against you. Using modern engineering tools well is normal software development.

The trade: you are fully responsible for every line of code and every technical decision in your submission.

You should be able to explain, in the technical interview:

- How your implementation works
- Why your database solution is safe
- What your tests protect
- Why you chose your architecture
- How authorization works
- What you accepted or rejected

17 AI Usage Documentation

Create `AI_LOG.md`. We do not want your full conversation history — four short sections are enough. If you did not use AI, simply say so.

01

Tools used

Which assistants, briefly.

03

Suggestions you rejected

At least one you disagreed with or significantly changed, and why.

02

How you used them

Exploration, planning, test generation, debugging, review, concurrency discussion.

04

Generated code you modified

What it produced, what was insufficient, what you changed, why.

18 Git Practices

Treat the assessment as normal professional development work. We are interested in how your implementation evolved, so avoid a single `final` commit unless there is a specific reason.

```
feat: add task assignment domain logic
feat: implement task activity history
fix:  enforce project membership on task mutations
fix:  make project task numbering concurrency-safe
test: add task authorization regression coverage
```

19 README

Update or create `README.md` with enough information for another engineer to run and evaluate your work.

- **Setup** — how to install dependencies
- **Environment** — required variables, no real secrets
- **Database** — how to initialize and run it
- **Running** — backend and frontend
- **Tests** — how to run the suite
- **Technical Decisions** — the major ones
- **Known Limitations** — what you left out

Provide a `.env.example` where appropriate. Never commit real credentials.

20 Developer Experience

We should be able to clone your repository and run the application without reverse-engineering your environment.

Docker is welcome, not required. A `docker compose up` setup is fine where it fits, but only mandatory if the starter repository already requires it.

Whatever approach you take must be clearly documented and reproducible on a clean machine.

PART FOUR

Evaluation and Submission

What we actually assess, the judgment we expect you to exercise, and everything the final submission has to contain.

21 What We Evaluate

The assessment measures more than whether the feature works.

- Understanding an unfamiliar codebase
- Requirement interpretation
- Backend engineering
- Frontend engineering
- Database design
- Security and authorization
- Data consistency
- Concurrency handling
- API design
- TypeScript usage
- Testing strategy
- Debugging ability
- Code organization
- Git practices and documentation
- Recognizing unnecessary complexity

22 Judgment and Restraint

AMBIGUITY

You are allowed to decide

Some requirements are deliberately unspecified. Engineers work with requirements that need judgment.

- Make a reasonable assumption
- Document the assumption
- Continue — do not block on minor ambiguity

COMPLEXITY

Avoid overengineering

Architectural improvements are welcome when they solve a real problem. Adding technology to look sophisticated is not.

You do not automatically need Kafka, RabbitMQ, Redis, microservices, CQRS, event sourcing or Kubernetes.

The question we will ask: what concrete problem does this solve in the current system?

23 Final Reflection

End `ASSESSMENT_NOTES.md` with a section called `## If I Had Two More Days` .

- **Explain what you would improve** given two additional working days.
- **Prioritize it.** The ordering is the point.
- **Why we ask:** what you consider most important after finishing tells us more than the implementation alone.

24 Submission Checklist

INCLUDE

Your submission contains

- Application source code
- `README.md`
- `ASSESSMENT_NOTES.md`
- `BUG_REPORT.md`
- `AI_LOG.md`
- Automated tests
- Database / schema changes
- `.env.example` where applicable

VERIFY

Before you submit

- The application starts successfully
- The requested feature works
- Tests run successfully
- Schema changes are included
- No credentials or secrets committed
- Documentation is complete
- You can explain your implementation

25 How to Submit

WHERE

The careers portal

Submit at engtechnos.com/en/careers/submit-task.

- **Email Address *** — the address you applied with
- **Job Position *** — select the role you applied for
- **GitHub Repository URL** — your repository, built on the starter repo
- **Live Demo URL** — optional, and it counts in your favour
- **Deliverable Files** — up to 5, ZIP/PDF/PNG/JPG, 10MB each
- **Additional Notes** — put your approximate time spent here

Hosting costs money, so a deployment is never required and skipping it costs you nothing. A working live URL is a genuine advantage. Never include passwords or production credentials.

THE FORM

What you will see

Email Address *

e.g. john@example.com

Job Position *

Full-Stack Developer Intern

GitHub Repository URL

e.g. https://github.com/username/repo

Live Demo URL

e.g. https://my-project.vercel.app

Deliverable Files (You can upload up to 5 files)

Drag & drop your file here, or click to browse
ZIP, PDF, PNG, JPG - Max 10MB

Additional Notes

Any additional notes about your submission...

Submit Task

26 Final Note

There is no single expected implementation. Different engineering decisions may all be valid when they are technically sound and appropriately justified.

What we are primarily interested in: understand → reason → implement → verify → communicate — rather than simply producing the largest amount of code.

Good luck.